

GRAND CHALLENGE MONOGRAPH COLLECTION

TYPE THEORY

The Grand Unified Theory of Computation

PROTOCOL

Computation as Communication

Volume IV

Grand Challenge Labs

Scope and series thesis

The phrase “grand unified theory” names a research thesis, not an established theorem. This volume asks what changes when the primary computational object is interaction rather than closed function evaluation.

The present Gate-1 teaching kernel develops only a finite synchronous binary session calculus. It establishes the vocabulary and executable evidence needed for later chapters. It does not establish arbitrary deadlock freedom, fairness, fault tolerance, security, asynchronous correctness, or distributed-system correctness.

How to read the formalism

A session type is an ordered communication obligation. A typing judgment

$$\Gamma; \Delta \vdash P$$

separates reusable value assumptions in Γ from linear channel obligations in Δ . The distinction is essential: the calculus is designed so that two parallel subprocesses cannot independently consume the same linear endpoint obligation.

Contents

1	A Program With Someone on the Other End	1
1.1	From values to traces	1
1.2	The smallest session language	2
1.3	Processes	3
1.4	Worked trace	4
1.5	Computational laboratory	4
1.6	Do not confuse: the protocol with the meaning	5
1.7	Exercises	5
2	Two Ends of One Obligation	7
2.1	Complementary views	7
2.2	Paired endpoints	8
2.3	Linearity: one obligation, one owner in the derivation	9
2.4	What duality prevents	10
2.5	Safety is not enough	11
2.6	Computational laboratory	12
2.7	Exercises	12

Chapter 1

A Program With Someone on the Other End

A closed function is patient. It can wait until all of its inputs are available, reduce internally, and eventually return a value. A conversation cannot be described that way without losing the order in which information becomes available.

Consider a client and a service. The client first sends a natural number. The service then replies with a Boolean. The interaction is not merely associated with the payload types $\mathbb{N}$ and Bool . It also contains an order:

send $\mathbb{N}$ then receive Bool then close.

Governing Principle

For an interactive computation, “what may happen next” is part of the type-level obligation. A protocol type is therefore not only a description of data. It is a description of admissible continuations.

1.1 From values to traces

Suppose an ordinary function has type

$$f : \mathbb{N} \rightarrow \text{Bool}.$$

The type says that a natural-number input is mapped to a Boolean result. It does not say that another independently executing party must be ready at an intermediate point.

Now consider the finite interaction

$$C \xrightarrow{42} S \quad C \xleftarrow{\text{true}} S.$$

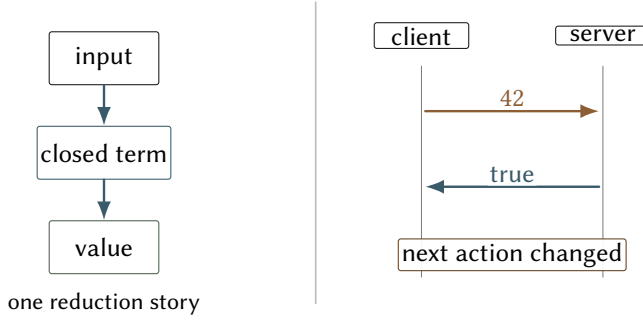


Figure 1.1: Plate 1. Closed function, open conversation. A closed term is organized around producing a value; an interaction trace advances as another participant acts. The contrast is pedagogical: it does not assert that every computation is purely closed or purely interactive.

The arrows are events. After the first event, both peers are in a new protocol state. The next permitted action is different from the action permitted before the first event.

Do not confuse

A trace is not a function value written sideways. A protocol can remain open indefinitely, can branch on messages from another party, and can depend on scheduling. Closed evaluation and open interaction have different semantic burdens.

1.2 The smallest session language

For the first executable kernel, payload values are drawn from a small ordinary value language. Session types are generated by

$$S ::= !A.S \mid ?A.S \mid \text{end}! \mid \text{end}?$$

The intended reading is directional from the endpoint whose type is being written.

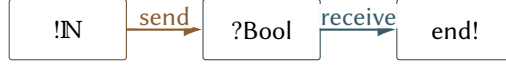
Definition 1.1 (Output session). $!A.S$ requires the endpoint to send one value of payload type A , then continue according to S .

Definition 1.2 (Input session). $?A.S$ requires the endpoint to receive one value of payload type A , then continue according to S .

Definition 1.3 (Closing endpoints). $\text{end}!$ requires active close; $\text{end}?$ requires the peer-facing wait for close. They are kept distinct so that endpoint obligations remain complementary.

The client protocol above is therefore

$$S_C = !N.? \text{Bool}.\text{end}!$$



current constructor = permitted action
 continuation = next protocol state

Figure 1.2: Plate 2. A protocol is a typed state machine. The finite session $!N.?Bool.end!$ records ordered communication obligations, not merely a set of payload types. The picture covers the synchronous finite teaching fragment only.

The protocol is an ordered object. Swapping the first two constructors changes the protocol.

Intuition

Read a session type from left to right as a program counter for communication. Each constructor says which action is legal at the current frontier and which type governs the continuation after that action is discharged.

1.3 Processes

The full Volume-IV preflight defines a richer process language. In this chapter we need only output, input, close, wait, and inaction:

$$P ::= 0 \mid x!\langle v \rangle.P \mid x?(y).P \mid \text{close } x.P \mid \text{wait } x.P.$$

The process

$$x!\langle 42 \rangle.x?(b).\text{close } x.0$$

has exactly the visible protocol shape of S_C , assuming $42 : \mathbb{N}$ and $b : \text{Bool}$.

Formal Interlude

A process typing judgment has the form

$$\Gamma ; \Delta \vdash P.$$

The value context Γ contains ordinary reusable assumptions such as $b : \text{Bool}$. The channel environment Δ contains endpoint obligations such as $x : !N.?Bool.end!$. At Gate 1 we use this judgment only for local derivations. Parallel composition and environment splitting are introduced in Chapter 2.

A schematic output rule is

$$\frac{\Gamma \vdash v : A \quad \Gamma ; \Delta, x : S \vdash P}{\Gamma ; \Delta, x : !A.S \vdash x!\langle v \rangle.P}.$$

The premise for the continuation uses $x : S$, not the original output type. Communication therefore consumes an obligation.

Likewise, a schematic input rule is

$$\frac{\Gamma, y : A; \Delta, x : S \vdash P}{\Gamma; \Delta, x : ?A.S \vdash x?(y).P}.$$

1.4 Worked trace

Worked Example

Take

$$S_C = !N.?Bool.end!.$$

The event sequence

`send : N,    recv : Bool,    close`

consumes the constructors of S_C in order and reaches the terminal state.

By contrast,

`recv : Bool,    send : N,    close`

uses exactly the same two payload types but violates the protocol at the first step.

This is the first lesson that session types add beyond an ordinary product or function type: *ordering is type-relevant*.

1.5 Computational laboratory

Computational Laboratory

Run

`python labs/lab01_protocol_trace.py.`

The laboratory parses finite protocol descriptions, checks traces action-by-action, and runs positive and hostile fixtures. It deliberately rejects wrong order, wrong payload type, premature termination, and trailing events.

The implementation uses a deterministic trace validator because reproducible evidence is useful. The mathematical protocol language is not thereby defined to have only one scheduler.

1.6 Do not confuse: the protocol with the meaning

Imagine a payment service whose protocol requires

$$\text{send} : \mathbb{N}; \text{recv} : \text{Bool}.$$

A client may send 0 where the business rule requires a positive amount. The communication can be protocol-correct while the application is wrong.

Do not confuse

Session fidelity constrains communication shape. It does not by itself establish business invariants, security properties, persistence, fault tolerance, or the semantic truth of payloads. Those are separate proof obligations.

1.7 Exercises

Checkpoint

- 1.1 Translate “send a natural number, receive a Boolean, then close” into the Gate-1 session notation.
- 1.2 Explain why $!N.?Bool.end!$ and $?Bool.!N.end!$ are different protocol types.
- 1.3 State the role of Γ and the role of Δ in $\Gamma; \Delta \vdash P$.

Core

- 1.4 Write a process skeleton that follows $!Bool.!N.end!$.
- 1.5 Give a valid action trace for $?N.?Bool.end?$.
- 1.6 Give two distinct invalid traces for $!N.?Bool.end!$, one with the wrong order and one with the wrong payload type.

Synthesis

- 1.7 Compare the information contained in a function type $\mathbb{N} \rightarrow \text{Bool}$ with the session type $!N.?Bool.end!$. Name one fact each type expresses that the other does not.
- 1.8 Explain why a deterministic trace checker can be valid evidence for finite examples without turning the mathematical process semantics into a deterministic semantics.

Proof Workshop

- 1.9 Prove by inspection of the four constructors that every finite Gate-1 session type has a well-defined first communication action.
- 1.10 Give an induction principle for finite Gate-1 session types and use it to prove that the number of actions in a complete trace equals the number of session constructors consumed.

Design Clinic

- 1.11 Extend the trace syntax with a payload-value predicate, such as “positive natural”, without claiming that ordinary session fidelity already enforces it. State what new checking layer you added.

Challenge

- 1.12 A service is intentionally nonterminating: it repeatedly receives a request and sends a reply. Explain why “does not normalize to a final value” is not by itself a correctness defect. State which additional formal machinery is needed before this example belongs to the Volume-IV calculus.

Chapter 2

Two Ends of One Obligation

A protocol becomes useful only when two endpoints can meet. If one endpoint promises to send a natural number, the peer must be prepared to receive one. If one endpoint selects a branch, the peer must offer that branch. The relation connecting the two endpoint views is **session duality**.

2.1 Complementary views

For the finite Gate-1 language, duality is defined structurally:

$$\begin{aligned} \overline{!A.\bar{S}} &= ?A.\bar{S}, & \overline{?A.\bar{S}} &= !A.\bar{S}, \\ \overline{\text{end}!} &= \text{end?}, & \overline{\text{end?}} &= \text{end}!. \end{aligned}$$

Thus

$$\overline{!N. ?\text{Bool}. \text{end}!} = ?N. !\text{Bool}. \text{end?}.$$

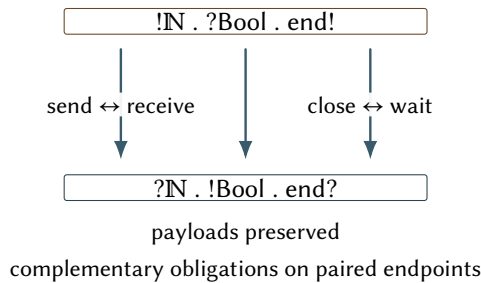


Figure 2.1: Plate 3. Two ends, complementary actions. Session duality swaps sending with receiving, selecting with offering, and active close with passive wait. This operation is complementary protocol behavior; it is not logical negation in general.

Do not confuse

Duality is not a generic synonym for logical negation. It is an operation on a particular session syntax that swaps complementary communication actions. Chapter 13 later studies proof/process correspondences under explicit translations; no such unrestricted identification is assumed here.

Theorem 2.1 (T4.1, finite duality involution). *For every finite Gate-1 session type S ,*

$$\overline{\overline{S}} = S.$$

Proof. By structural induction on S . If $S = \text{end}!$ or $S = \text{end}?$, the claim follows from the two terminal clauses. If $S = !A.T$, then

$$\overline{\overline{!A.T}} = \overline{?A.\overline{T}} = !A.\overline{\overline{T}} = !A.T$$

by the induction hypothesis. The input case is symmetric. $\square$

The theorem is small, but it carries a critical burden: it justifies treating duality as a two-sided endpoint transformation for this finite syntax.

2.2 Paired endpoints

We write

$$(\nu x y)P$$

for a restriction that creates paired endpoints x and y inside P . If x is governed by S , then y is governed by $\overline{S}$.

Consider

$$(\nu x y)(x!\langle 42 \rangle.x?(b).\text{close } x.0 \mid y?(n).y!\langle \text{true} \rangle.\text{wait } y.0).$$

The first visible pair of prefixes are complementary. A synchronous reduction can transfer the value 42 from the sender to the receiver:

$$\begin{aligned} &(\nu x y)(x!\langle 42 \rangle.P \mid y?(n).Q) \\ &\rightarrow (\nu x y)(P \mid Q[42/n]). \end{aligned}$$

Formal Interlude

The process reduction is accompanied by evolution of the channel environment. If the pre-step environment contains

$$x : !A.S, \quad y : ?A.\overline{S},$$

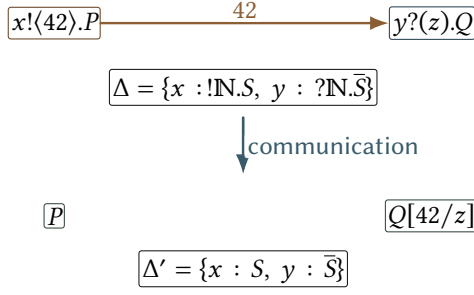


Figure 2.2: Plate 4. A handshake consumes obligations. A principal synchronous communication advances both process continuations and the session environment from Δ to Δ' . One commuting step is not a proof of global liveness.

then after communication the corresponding obligations are

$$x : S, \quad y : \bar{S}.$$

We write this abstractly as

$$\Delta \Rightarrow \Delta'.$$

The later subject-reduction theorem will state that process reduction and environment evolution remain synchronized.

At Gate 1 we prove only principal-redex compatibility. The full subject-reduction induction is a Chapter-3 obligation.

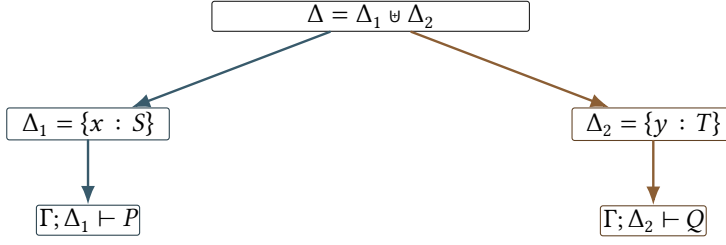
Proposition 2.2 (Principal send/receive compatibility). *Suppose the restricted endpoints x, y carry dual finite session types whose first actions are $!A.S$ and $?A.\bar{S}$, and suppose the sent value has payload type A . Then the principal synchronous communication step advances the endpoint obligations to S and $\bar{S}$.*

Proof. Immediate from the output/input typing premises, the duality definition, and the principal communication reduction. The receiver continuation substitutes a value already typed at A ; the remaining channel obligations are exactly the continuations recorded by the two session constructors. This proposition intentionally does not yet claim preservation under arbitrary evaluation contexts or structural congruence. $\square$

2.3 Linearity: one obligation, one owner in the derivation

A process may use parallel composition:

$$P \mid Q.$$



disjoint domains: one endpoint obligation
cannot be spent by both subprocesses

Figure 2.3: Plate 5. Linear splitting of channel authority. The parallel rule partitions channel obligations between subprocesses, preventing one formal endpoint obligation from being independently consumed twice. Formal linearity does not establish unique physical ownership after compilation or deployment.

To keep a linear endpoint from being independently consumed by both sides, the channel environment is split:

$$\frac{\Gamma; \Delta_1 \vdash P \quad \Gamma; \Delta_2 \vdash Q \quad \Delta = \Delta_1 \uplus \Delta_2}{\Gamma; \Delta \vdash P \mid Q}.$$

The disjoint union symbol matters. It means that a linear channel binding belongs to at most one premise.

Proposition 2.3 (T4.2, disjoint linear splitting). *If $\Delta = \Delta_1 \uplus \Delta_2$, then no endpoint binding occurs in both Δ_1 and Δ_2 . Conversely, the union reconstructs exactly the bindings of Δ .*

Proof. This is a property of the finite-map splitting relation used by ‘PROTO-0’. The derivation of a split is built only from assigning each distinct binding to exactly one side. Induction on that construction gives disjointness and reconstruction. $\square$

Do not confuse

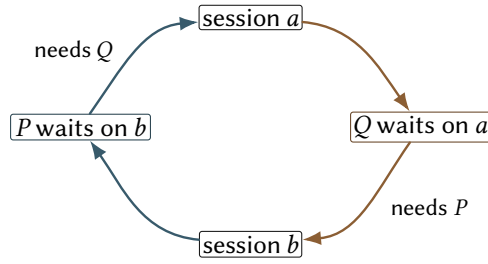
A linear typing derivation controls formal use of an endpoint name. It does not prove that a compiler, operating system, broker, network stack, or physical transport maintains unique ownership. The implementation theorem would require an additional compilation/runtime model.

2.4 What duality prevents

Two peers can have the same payload vocabulary and still be incompatible.

For example,

$$S_1 = !\mathbb{N}.? \text{Bool}.end!, \quad S_2 = !\mathbb{N}.? \text{Bool}.end!.$$



locally compatible; globally no enabled handshake

Figure 2.4: Plate 6. Safe but stuck. Two locally well-typed sessions can participate in a cyclic wait when processes acquire them in incompatible orders. The witness refutes unrestricted progress from local session typing; it does not refute session fidelity.

Both attempt to send first. They are not dual. Under the synchronous semantics there is no principal send/receive redex between them.

The dual peer is instead

$$\overline{S}_1 = ?N.!Bool.end?.$$

Worked Example

The laboratory compares peers structurally rather than by checking only their payload multiset. It therefore rejects the pair

$$!N.?Bool.end! \quad \text{and} \quad !N.?Bool.end!,$$

even though both mention exactly one N and one $Bool$.

2.5 Safety is not enough

It is tempting to jump from “the endpoints are dual” to “the whole distributed program cannot get stuck.” That inference is false once a process can interleave several sessions.

Suppose one process waits on channel x before sending on u , while another waits on v before sending on y , and the restrictions pair x with y and u with v . Each individual endpoint can be assigned a locally compatible session type, yet the network can form a cycle of waits with no enabled communication.

This is the most important negative boundary in the volume.

Governing Principle

Session fidelity is a local type-preservation property. Global liveness requires additional structure or hypotheses. Do not obtain deadlock freedom by renaming fidelity.

The full counterexample is developed in Chapter 6. Gate 1 includes the plate now because the reader must know from the beginning that the volume will not hide this distinction.

2.6 Computational laboratory

Computational Laboratory

Run

```
python labs/lab02_duality_checker.py.
```

The laboratory parses finite sessions, computes duals, verifies involution over a fixture set, accepts exact dual peers, and rejects same-direction or label-incompatible peers.

The checker is executable evidence for the finite syntax. It is not a proof of every theorem in the later recursive, asynchronous, or multiparty calculi.

2.7 Exercises

Checkpoint

- 2.1 Compute $\overline{!N.?Bool.end!}$.
- 2.2 State the dual of $end!$ and explain why using one undirected terminal constructor would hide a protocol obligation in this teaching calculus.
- 2.3 In one sentence, distinguish session duality from logical negation.

Core

- 2.4 Write the two endpoint types for a protocol in which the client receives a Boolean and then sends a natural number before the client waits for close.
- 2.5 Give a principal synchronous reduction for $(\nu x y)(x!\langle 7 \rangle.P \mid y?(n).Q)$.
- 2.6 Explain why the parallel typing rule uses $\Delta_1 \uplus \Delta_2$ rather than ordinary union.

Synthesis

- 2.7 Compare “dual endpoints” with “deadlock-free network.” Give a concrete reason the first does not imply the second in a multi-session setting.
- 2.8 Compare formal linearity of an endpoint variable with unique ownership of a real socket. List two implementation assumptions needed to connect them.

Proof Workshop

- 2.9 Prove T4.1 for the finite Gate-1 syntax using structural induction.
- 2.10 Prove that if a finite channel environment is split by assigning every binding to exactly one of two subenvironments, then the two subenvironments are disjoint and reconstruct the original.

Design Clinic

- 2.11 Design an error report for a duality checker that distinguishes payload mismatch, direction mismatch, and premature termination. Explain why one generic “protocol mismatch” message loses useful evidence.

Challenge

- 2.12 Construct a two-session cyclic wait in prose or process pseudocode. Assign locally compatible session actions to every endpoint, then identify the wait-for cycle. State precisely which property fails and which local safety property can remain intact.

Gate-1 transition

The first two chapters establish the smallest executable protocol kernel. The next chapter must prove that a well-typed communication step advances process state and session state together. Later chapters will then deliberately construct a locally well-typed multi-session deadlock to separate session fidelity from global liveness.

Gate-1 notation guide

Notation	Meaning in this volume
$!A.S$	send a value of payload type A , then continue as S
$?A.S$	receive a value of payload type A , then continue as S
$\oplus\{l_i : S_i\}$	internally select one labeled continuation
$\&\{l_i : S_i\}$	externally offer labeled continuations
$\bar{S}$	protocol dual of session type S
$\Gamma; \Delta \vdash P$	process P is typed under value context Γ and linear channel environment Δ
$P \rightarrow P'$	one process reduction step
$P \equiv_p Q$	selected process structural congruence relation
$\Delta \Rightarrow \Delta'$	protocol-obligation environment evolves with communication

Index

session duality, 7